Software Design Document: AITODO

1. Introduction

1.1 Project Objective

The AITODO project aims to develop an intelligent task management application for Android that transcends the limitations of traditional "passive recording" tools. The core objective is to create an "active thinking" task companion that leverages Artificial Intelligence to help users parse vague goals into actionable tasks, suggest schedules, and proactively identify to-do items from natural language conversations.

1.2 Scope

AITODO is a mobile application built using modern Android development practices. It integrates OpenAI-compatible APIs to provide conversational AI capabilities while maintaining strict user privacy through a "Bring Your Own Key" (BYOK) model. The application covers the full lifecycle of task management, from AI-assisted ideation and structured proposal acceptance to manual CRUD operations and persistence.

2. Requirements Specification

2.1 Functional Requirements

- **Core Task Management**:
  - Create, edit, delete, and archive tasks.
  - Support for task attributes: Title, Deadline (Date & Time), Reminders, and Repeat Modes.
  - State management: Mark tasks as completed, pin important tasks, and select multiple tasks for batch actions.
- **AI-Driven Features**:
  - **Conversational Interface**: A dedicated chat screen for interacting with AI models.
  - **Intelligent Task Parsing**: Automatically decompose macro goals or vague intentions into specific, executable TaskDraft objects.
  - **Task Proposal System**: A "Review -> Accept/Ignore" flow for AI-suggested tasks to ensure user control.
  - **Proactive Suggestions**: AI identification of potential tasks based on chat context.
- **Organization & Search**:
  - Filtering tasks by status (All, Current, Pinned, History).
  - Sorting by Title, Creation Date, or Deadline (with ascending/descending toggle).

    - Real-time search functionality with an expandable interface.

2.2 User Requirements

- **Data Privacy**: Users must be able to configure their own AI providers and API keys (BYOK), ensuring their data is not tied to a centralized service.
- **Efficiency**: Minimalist interaction patterns (e.g., swipe-to-pin, double-tap-to-edit) to reduce the time spent on manual management.
- **Structured Output**: AI responses must be reliably parsed into the application's internal data structures (JSON enforcement).

2.3 Non-Functional Requirements

- **Responsiveness**: The UI must provide instant feedback via StateFlow and Unidirectional Data Flow (UDF).
- **Offline Capability**: Core task management and data persistence must work without an internet connection (using local DataStore).
- **Modern UX**: Implementation of Material3 design principles, smooth animations, and haptic feedback for key gestures.

3. Overall Design

3.1 Architectural Overview

AITODO follows the **MVVM (Model-View-ViewModel)** architectural pattern combined with a **Repository** pattern for data abstraction.

- **UI Layer (View)**: Built entirely with **Jetpack Compose**. It observes StateFlow from ViewModels to render the UI and sends user events back to the ViewModels.
- **Logic Layer (ViewModel)**: Manages UI state, handles business logic (filtering, sorting, AI logic), and interacts with repositories.
- **Data Layer (Model/Repository)**:
    - AISettingsRepository: Manages AI provider configurations and chat settings using **DataStore Preferences**.
    - TaskRepository: (Planned) To manage task persistence.
    - OpenAIService: Retrofit interface for external AI communication.

3.2 AI Integration Model

The application adopts an "Open API" model. Instead of binding to a specific model, it allows users to configure:

- Base URL (OpenAI-compatible)
- API Key
- Model ID
- System Prompts and Temperature

Communication is handled via **Retrofit** with structured JSON output requirements (Tool Use/Function Calling) to ensure compatibility between LLM responses and the TaskDraft domain model.

4. User Interface Design

4.1 Key Screens

- **TaskListScreen**: The main dashboard showing filtered tasks (Current, Pinned, etc.). It features a reactive Top Bar with search, sort, and filter controls.
- **AIChatScreen**: A conversational interface for interacting with the AI. It displays message history and a dedicated "Draft Area" where AI-proposed tasks are listed for user review.
- **Settings Hub**: A nested navigation system including:
  - **SettingsScreen**: Main entry point for AI and General settings.
  - **ProviderListScreen**: Management of multiple AI service configurations.
  - **ProviderEditScreen**: Form for detailed configuration and model fetching.

4.2 Reusable Components

- **TaskItem**: A swipeable card component supporting "Swipe Right to Pin" with haptic feedback and "Double-tap to Edit" gestures.
- **AddTaskDialog**: A compact, context-aware dialog used for both creating new tasks and editing existing ones.
- **CompactPickers**: Custom-built Date and Time pickers optimized for dialog use, ensuring usability on smaller mobile screens.

4.3 Interaction Patterns

- **Gesture-Driven Actions**: Utilization of pointerInput and combinedClickable to handle sophisticated gestures like double-taps and swipes without conflicting with scroll behaviors.
- **Expandable Search**: A Material3 Search Bar that dynamically adjusts the UI state when focused, providing a seamless transition between viewing and searching.

5. Key Technologies

5.1 Modern Android Stack

- **Kotlin**: The primary language, leveraging Coroutines for asynchronous work.
- **Jetpack Compose**: Used for building a declarative, state-driven UI.
- **Material3**: The latest design language for consistent look and feel.
- **Navigation Compose**: Handles screen transitions and deep linking.

5.2 Networking & Persistence

- **Retrofit & OkHttp**: For robust API communication with AI providers.
- **Gson**: For serializing and deserializing JSON data, particularly complex AI tool calls.
- **DataStore Preferences**: For lightweight, thread-safe persistence of user settings and configuration.

5.3 AI Implementation Logic

- **Tool Support**: Implementation of the propose_tasks tool definition, allowing the AI to return structured task data alongside natural language.
- **Middleware (TaskDraft)**: An intermediate data layer that validates and prepares AI suggestions before they are committed as TaskEntity objects.
- **Window Insets Handling**: Use of imePadding() and statusBarsPadding() to ensure the UI gracefully adjusts to system elements like the soft keyboard.

6. Testing and User Experience Analysis

6.1 Automated Cloud Testing

The application underwent comprehensive automated testing using **Firebase Robo Test**.

- **Results**: 100% success rate across 11 diverse Android devices.
- **Coverage**: The tests verified UI stability, screen transitions, and core interaction loops (Task creation, Screen navigation).

6.2 User Experience (UX) Feedback & Iteration

Based on user feedback and internal analysis, several iterative improvements were made:

- **Keyboard Management**: Resolved issues where the soft keyboard blocked input fields in the chat screen using imePadding().
- **State Feedback**: Added CircularProgressIndicator to provide visual cues during asynchronous AI processing.
- **Data Synchronization**: Optimized StateFlow subscriptions to ensure task additions via AI are immediately reflected in the main list.
- **Safety Checks**: Implemented "Dirty Data" checks and secondary confirmation popups in the editing flow to prevent accidental data loss.
- **AI Robustness**: Strengthened System Prompts and implemented a parsing retry mechanism to handle non-compliant AI output formats.

7. Conclusion

7.1 Summary of Achievements

AITODO successfully demonstrates the feasibility of a privacy-focused, AI-integrated task manager for Android. By combining a modern declarative UI (Compose) with an open AI

architecture (BYOK), the project provides a highly flexible and intelligent tool for productivity. The integration of structured AI output into a standard MVVM architecture serves as a robust template for future AI-driven mobile applications.

7.2 Challenges Overcome

- **Gesture Conflict Resolution**: Balancing complex gestures (swipe, double-tap, long-press) within a scrollable list.
- **Non-Deterministic AI Output**: Developing a robust parsing and review layer (TaskDraft) to bridge the gap between LLM creativity and application data integrity.

7.3 Future Outlook

- **On-Device AI**: Integration of lightweight models (e.g., Gemini Nano) to enable core AI features without an internet connection.
- **Productivity Analytics**: Utilizing AI to analyze historical task data and generate insights into personal productivity trends.
- **Enhanced Tooling**: Expanding AI capabilities to include automatic rescheduling and priority conflict resolution.